

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

CS 401/MCS 401 Lecture 20
Computer Algorithms I
Jan Verschelde, 3 August 2026

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

introduction

Important definitions needed for this lecture follow.

- 1 An algorithm is efficient if its worst case running time is bounded by a polynomial in the input size.
- 2 $\mathcal{P}$ collects all problems for which an efficient algorithm exists.
- 3 A certifier verifies an output of a problem.
- 4 $\mathcal{NP}$ collects all problems for which an efficient certifier exists.

We proved the following theorem.

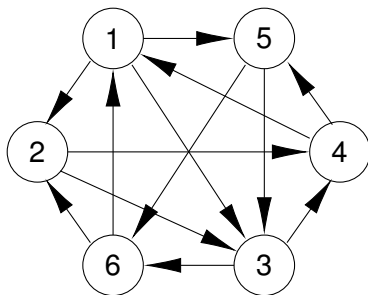
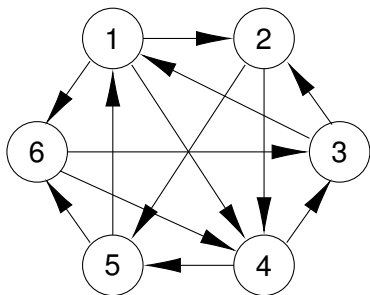
*If X is an NP-complete problem,
then X is solvable in polynomial time if and only if $\mathcal{P} = \mathcal{NP}$.*

To prove a problem is NP-complete,
we must show how it could encode *any* problem in $\mathcal{NP}$.

Definition (1)

Given a directed graph $G = (V, E)$, a cycle C in G is a *Hamiltonian cycle* if C visits each vertex exactly once.

Consider the directed graph at the left:



The labeling of the vertices at the right shows a Hamiltonian cycle.

computing Hamiltonian cycles

Topological ordering the vertices in a directed graph can be applied to detect the existence of a cycle.

Given a directed graph $G = (V, E)$,
compute a cycle C which visits every $v \in V$ exactly once.

Exercise 1: Consider the computation of a Hamiltonian cycle.

- 1 develop an algorithm to find a Hamiltonian cycle, and
- 2 what is the worst case running time of your algorithm?

the Hamiltonian Cycle Problem

The Hamiltonian Cycle Problem is:

Does a given directed graph contain a Hamiltonian cycle?

We are searching over the set of all permutations of the vertices.

This is a type of *sequencing problem*.

We will show that this problem is NP-complete.

First we have to show that Hamiltonian Cycle Problem is in $\mathcal{NP}$.

strategy to show a new problem is NP-complete

Given a new problem X , to show X is NP-complete:

- 1 Prove that $X \in \mathcal{NP}$.

This consists in exhibiting an efficient certifier algorithm.

- 2 Choose a problem Y that is known to be NP-complete.

- 3 Prove that $Y \leq_P X$.

The reformulation $Y \leq_P X$ implies that

- ▶ we can solve Y by calling the black box solver for X , and
- ▶ a polynomial number of elementary computational steps.

the Hamiltonian cycle problem is in $\mathcal{NP}$

Lemma (2)

The Hamiltonian Cycle Problem is in $\mathcal{NP}$.

Proof. For a directed graph $G = (V, E)$, a certificate that there is Hamiltonian cycle is a sequence of the vertices, written in the order of traversing the cycle.

Given a sequence of vertices, the certifier algorithm

- 1 verifies that every vertex is contained exactly once, and
- 2 for every consecutive pair of vertices in the sequence, there is an edge between the vertices in the pair.

For a graph with n vertices and m edges, the certifier algorithm runs in time $O(nm)$.

Q.E.D.

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- **representing assignments with cycles**
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

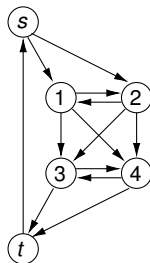
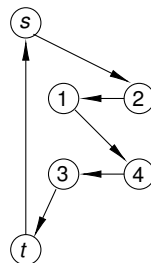
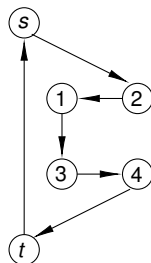
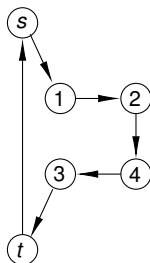
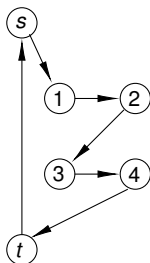
on the number of Hamiltonian cycles

For an instance of a problem Y that is known to be NP-complete, we will have to prove that $Y \leq_P X$, for an instance X of the Hamiltonian Cycle problem.

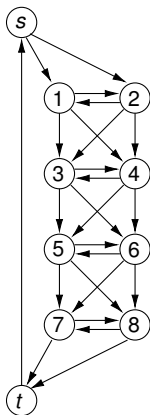
Before we choose a problem Y , it is useful to consider how much information we can store with a problem instance of the Hamiltonian Cycle problem.

In particular, how many Hamiltonian cycles can a graph have?

making graphs with many Hamiltonian cycles



a graph with 16 Hamiltonian cycles



s 1 2 3 4 5 6 7 8 t s
s 1 2 3 4 5 6 8 7 t s
s 1 2 3 4 6 5 7 8 t s
s 1 2 3 4 6 5 8 7 t s
s 1 2 4 3 5 6 7 8 t s
s 1 2 4 3 5 6 8 7 t s
s 1 2 4 3 6 5 7 8 t s
s 1 2 4 3 6 5 8 7 t s
s 2 1 3 4 5 6 7 8 t s
s 2 1 3 4 5 6 8 7 t s
s 2 1 3 4 6 5 7 8 t s
s 2 1 3 4 6 5 8 7 t s
s 2 1 4 3 5 6 7 8 t s
s 2 1 4 3 5 6 8 7 t s
s 2 1 4 3 6 5 7 8 t s
s 2 1 4 3 6 5 8 7 t s

defining a directed graph with 2^n cycles

Construct a directed graph $G = (V, E)$ as follows.

- The graph has a source s and a sink t as nodes: $s, t \in V$.
- Make an edge from sink t to source s , $(t, s) \in E$.
In the strict sense s is not a source and t is not a sink, but each cycle will start at the source s , go to the sink t , and return to the source via the edge (t, s) .
- Add $2n$ vertices $v_1, v_2, \dots, v_{2n}$ to V .
- $(s, v_1), (s, v_2), (v_{2n-1}, t), (v_{2n}, t) \in E$.
- For every odd numbered vertex: $(v_{2k+1}, v_{2k+2}), (v_{2k+2}, v_{2k+1}) \in E$, for $k = 0, 1, \dots, n-1$.
- For $k = 0, 1, \dots, n-2$: $(v_{2k+1}, v_{2k+3}), (v_{2k+1}, v_{2k+4}) \in E$ and $(v_{2k+2}, v_{2k+3}), (v_{2k+2}, v_{2k+4}) \in E$.

a directed graph with 2^n Hamiltonian cycles

Lemma (3)

For any n , we have a directed graph G with $2n + 2$ vertices and $6n + 1$ edges with 2^n Hamiltonian cycles.

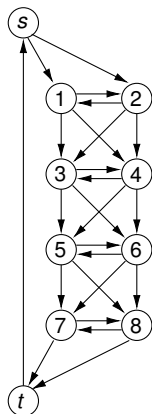
Proof. In the construction of the directed graph G , we count the number of vertices: $V = \{ s, t, v_1, v_2, \dots, v_{2n} \}$, $\#V = 2n + 2$, and the number of edges as $5 + 2n + 6n - 4 = 6n + 1$.

In G , there are n pairs of vertices v_{2k+1}, v_{2k+2} which can be traversed in forward and backward order in every cycle which starts from s , goes to t , and then comes back to s .

For each pair we have two choices, forward or backward. So, in total, there are 2^n Hamiltonian cycles in G .

Q.E.D.

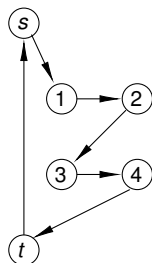
every Hamiltonian cycle represents an assignment



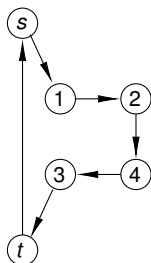
s 1 2 3 4 5 6 7 8 t s
 s 1 2 3 4 5 6 8 7 t s
 s 1 2 3 4 6 5 7 8 t s
 s 1 2 3 4 6 5 8 7 t s
 s 1 2 4 3 5 6 7 8 t s
 s 1 2 4 3 5 6 8 7 t s
 s 1 2 4 3 6 5 7 8 t s
 s 1 2 4 3 6 5 8 7 t s
 s 2 1 3 4 5 6 7 8 t s
 s 2 1 3 4 5 6 8 7 t s
 s 2 1 3 4 6 5 7 8 t s
 s 2 1 3 4 6 5 8 7 t s
 s 2 1 4 3 5 6 7 8 t s
 s 2 1 4 3 5 6 8 7 t s
 s 2 1 4 3 6 5 7 8 t s
 s 2 1 4 3 6 5 8 7 t s

$x_1 = 1, x_2 = 1, x_3 = 1, x_4 = 1$
 $x_1 = 1, x_2 = 1, x_3 = 1, x_4 = 0$
 $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 1$
 $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 0$
 $x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 1$
 $x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$
 $x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 1$
 $x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 0$
 $x_1 = 0, x_2 = 1, x_3 = 1, x_4 = 1$
 $x_1 = 0, x_2 = 1, x_3 = 1, x_4 = 0$
 $x_1 = 0, x_2 = 1, x_3 = 0, x_4 = 1$
 $x_1 = 0, x_2 = 1, x_3 = 0, x_4 = 0$
 $x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1$
 $x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 0$
 $x_1 = 0, x_2 = 0, x_3 = 0, x_4 = 1$
 $x_1 = 0, x_2 = 0, x_3 = 0, x_4 = 0$

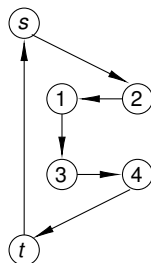
traversing Hamiltonian cycles and assignments



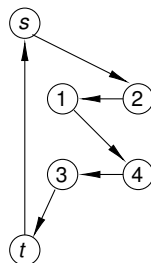
$$x_1 = 1$$
$$x_2 = 1$$



$$x_1 = 1$$
$$x_2 = 0$$



$$x_1 = 0$$
$$x_2 = 1$$



$$x_1 = 0$$
$$x_2 = 0$$

We have two paths:

- 1 P_1 : set $x_1 = 1$ if from 1 to 2, $x_1 = 0$ if from 2 to 1; and
- 2 P_2 : set $x_2 = 1$ if from 3 to 4, $x_2 = 0$ if from 4 to 3.

the 3-satisfiability problem is NP-complete

The 3-satisfiability problem, or 3-SAT for short is

Given $X = \{ x_1, x_2, \dots, x_n \}$ a set of Boolean variables, and $C_1, C_2, \dots, C_k$, a set of clauses over X , where each clause C_i has exactly three terms, does there exist a satisfying truth assignment?

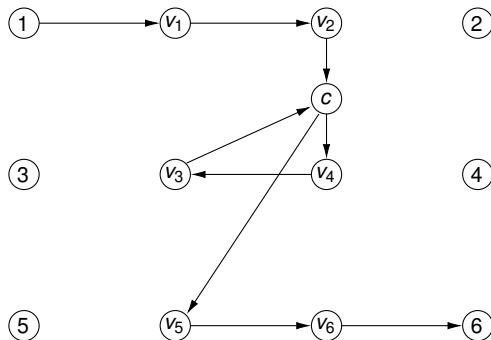
The example

$$(x \vee \bar{y} \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

is satisfied by $x = 1$, $\bar{y} = 1$, and $\bar{z} = 1$.

clauses and cycles

The clause $c = x_1 \vee \bar{x}_2 \vee x_3$ is represented by a cycle:



The clause is represented by the cycle that contains the path

$$v_1, v_2, c, v_4, v_3, c, v_5, v_6,$$

with corresponding assignment $x_1 = 1$, $x_2 = 0$, and $x_3 = 1$.

constructing cycles for every clause

Consider a sequence of clauses, each with three terms,

$$c_k = t_1 \vee t_2 \vee t_3, \quad t_1, t_2, t_3 \in \{x_1, \bar{x}_1, x_2, \bar{x}_2, \dots, x_n, \bar{x}_n\},$$

for $k = 1, 2, \dots, m$.

For each clause c_k , add to the graph G as follows.

- 1 Add the vertex c_k to V .
- 2 For each term t of the clause c_k ,
 - ▶ if $t = x_i$, add vertices $v_{k,i}, v_{k,i+1}$ and insert edges $(v_a, v_{k,i})$, $(v_{k,i}, v_{k,i+1})$, and $(v_{k,i+1}, v_b)$, in the path from v_{2i-1} to v_{2i} .
 - ▶ if $t = \bar{x}_i$, add vertices $v_{k,i}, v_{k,i+1}$ and insert edges $(v_b, v_{k,i+1})$, $(v_{k,i+1}, v_{k,i})$, and $(v_{k,i}, v_a)$, in the path from v_{2i} to v_{2i-1} .
- 3 Add edges to connect the vertex c_k to the end points of the edges corresponding to the terms of the clause c_k , as indicated in the picture on the previous slide.

an instance of a 3-SAT problem

Consider a sequence of clauses, each with three terms,

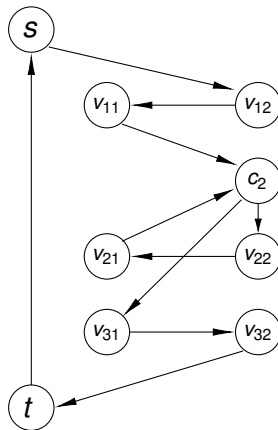
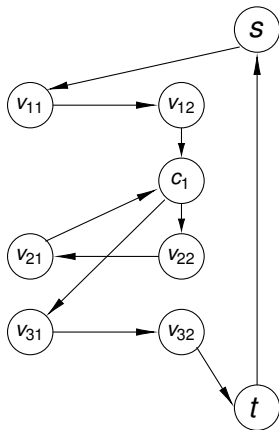
$$c_k = t_1 \vee t_2 \vee t_3, \quad t_1, t_2, t_3 \in \{x_1, \bar{x}_1, x_2, \bar{x}_2, \dots, x_n, \bar{x}_n\},$$

for $k = 1, 2, \dots, m$.

For every clause, 7 vertices and 7 edges are added to G .
The constructed directed graph G has $2n + 2 + 7m$ vertices
and $6n + 1 + 7m$ edges.

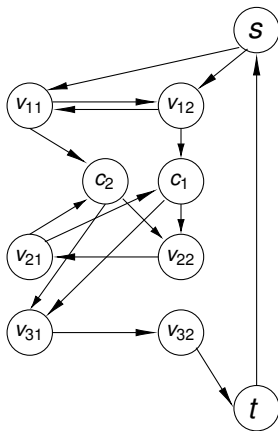
encoding $(x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$

$c_1 = x_1 \vee \bar{x}_2 \vee x_3$, $c_2 = \bar{x}_1 \vee \bar{x}_2 \vee x_3$, two separate clauses

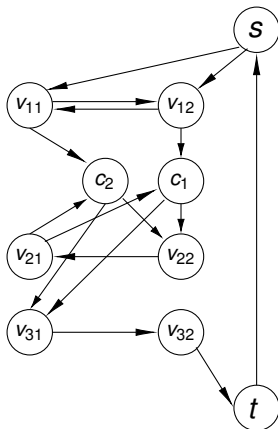


encoding $(x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3)$

$c_1 = x_1 \vee \bar{x}_2 \vee x_3$, $c_2 = \bar{x}_1 \vee \bar{x}_2 \vee x_3$, combined

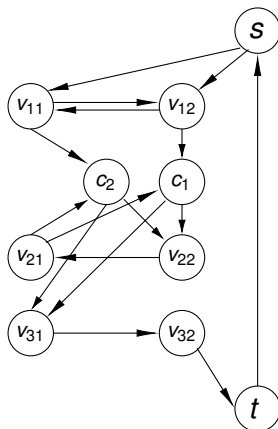


3-SAT and Hamiltonian cycle are equivalent problems



The cycle $s, v_{11}, v_{12}, c_1, v_{22}, v_{21}, c_2, v_{31}, v_{32}, t, s$ corresponds to $x_1 = 1, x_2 = 0$, and $x_3 = 1$. Observe the other cycle.

a polynomial-time reduction



Every clause adds 7 vertices and 7 edges, plus three edges to the next clause, one edge from s and one edge to t .

encoding a 3-SAT instance as Hamiltonian cycle

Exercise 2: Encode

$$(x \vee \bar{y} \vee z) \wedge (\bar{x} \vee \bar{y} \vee z) \wedge (x \vee y \vee \bar{z})$$

as a Hamiltonian cycle problem.

3-SAT and Hamiltonian cycles

Lemma (4)

An instance of a 3-SAT problem defined by m clauses c_k is satisfiable if and only if G contains a Hamiltonian cycle which visits every vertex c_k in G , for all $k = 1, 2, \dots, m$.

Proof. If and only if means $\Rightarrow$ and $\Leftarrow$.

$\Rightarrow$ If the 3-SAT instance is satisfiable,
then we have a satisfying truth assignment.

As we evaluate each clause at the assignment, we traverse the edges either forward or backward depending on whether we assign 1 or 0 to each variable, and encounter each node c_k in G corresponding to the clause c_k .

Thus we obtain a Hamiltonian cycle.

3-SAT and Hamiltonian cycles, proof continued

Lemma (4)

An instance of a 3-SAT problem defined by m clauses c_k is satisfiable if and only if G contains a Hamiltonian cycle which visits every vertex c_k in G , for all $k = 1, 2, \dots, m$.

Proof continued. We continue with the $\Leftarrow$ part.

- $\Leftarrow$ Traversing the Hamiltonian cycle, visiting each vertex c_k , we evaluate the corresponding clause c_k , with the assignments as defined by the forward or backward traversal order to the variables involved in the clause c_k .

As we visit each vertex c_k , and by construction of G , this leads to a satisfying truth assignment for the 3-SAT instance.

Q.E.D.

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- **Hamiltonian cycle is NP-complete**

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

Theorem (5)

The Hamiltonian Cycle Problem is NP-complete.

Proof. By Lemma (2),
the Hamiltonian Cycle Problem is in $\mathcal{NP}$.

As problem known to be NP-complete, we choose 3-SAT.

For any arbitrary instance of the 3-SAT problem with m clauses in n variables, we construct a directed graph G with $2n + 2n + 7m$ vertices and $6n + 1 + 7m$ edges in $O(n + m)$ time.

By Lemma (4), the black box solver for the Hamiltonian Cycle Problem on the constructed graph determines whether any arbitrary instance of the 3-SAT problem is satisfiable.

Q.E.D.

Sequencing Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

CS 401/MCS 401 Lecture 20
Computer Algorithms I
Jan Verschelde, 3 August 2026

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- **a decision problem**
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

introduction

- 1 An algorithm is efficient if its worst case running time is bounded by a polynomial in the input size.
- 2 $\mathcal{P}$ collects all problems for which an efficient algorithm exists.
- 3 A certifier verifies an output of a problem.
- 4 $\mathcal{NP}$ collects all problems for which an efficient certifier exists.

*If X is an NP-complete problem,
then X is solvable in polynomial time if and only if $\mathcal{P} = \mathcal{NP}$.*

To prove a problem X is NP-complete, we apply the strategy:

- 1 Prove that $X \in \mathcal{NP}$.
- 2 Choose a problem Y known to be NP-complete.
- 3 Prove that $Y \leq_P X$.

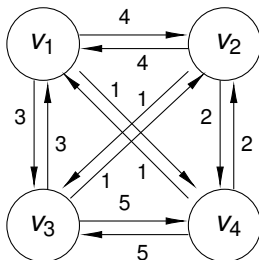
the traveling salesman problem

We consider a *weighted*, directed graph G as input:

- 1 The vertices represent cities which must all be visited.
- 2 For each pair of vertices u and v , we have distances $d(u, v)$ and $d(v, u)$, as weights on the edges.

The weights in the graph may be asymmetric, e.g.: $d(u, v) \neq d(v, u)$.

Find a tour with the minimum distance traveled in the graph:

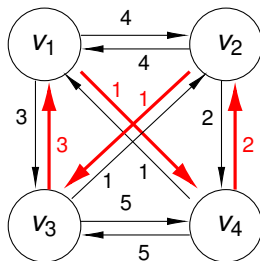
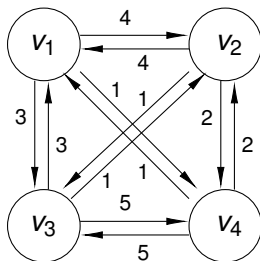


a decision problem

The optimization version of this problem ask for the tour of all cities with the minimum distance traveled.

The tour starts at the home city of the salesman and ends there. The decision version of the Traveling Salesman Problem is then

Given a weighted, directed graph G , and
a bound D on the distance traveled,
is there a tour in G of length at most D ?



solving the traveling salesman problem

The traveling salesman problem is an important problem.

Given a weighted, directed graph G , and a bound D , compute a tour in G of length at most D .

Exercise 3: Consider solving the traveling salesman problem.

- 1 Develop an algorithm to solve the traveling salesman problem.
- 2 What is the worst case running time of your algorithm?
- 3 Does the size of D help in reducing the running time?

Hamiltonian Cycle and Traveling Salesman Problems

The decision version of the Hamiltonian Cycle Problem is

Given a directed graph G ,
is there a cycle in G visiting each vertex exactly once?

The decision version of the Traveling Salesman Problem is

Given a weighted, directed graph G , and
a bound D on the distance traveled,
is there a tour in G of length at most D ?

We view Traveling Salesman as a generalization of Hamiltonian cycle.

traveling salesman is in $\mathcal{NP}$

Lemma (6)

The traveling salesman problem is in $\mathcal{NP}$.

Proof. The certificate for a solution lists the visited vertices in the tour. The certifier verifies that

- 1 the list of visited vertices represents a permutation, that is: every vertex in G occurs exactly once, and
- 2 the length of the distances on the tour is at most the bound D .

Both operations happen in $O(n)$,
in particular if an adjacency matrix is used to store the distances.

So we have an efficient certifier algorithm and
therefore the traveling salesman problem is in $\mathcal{NP}$.

Q.E.D.

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- **traveling salesman is NP-complete**
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

traveling salesman is NP-complete

Theorem (7)

The traveling salesman problem is NP-complete.

Proof. By Lemma (6), the problem is in $\mathcal{NP}$.

The NP-problem we choose is the Hamiltonian Cycle problem.

We have to prove that an arbitrary instance of the Hamiltonian Cycle problem can be solved

- by the black box solver for the Traveling Salesman problem, and
- a polynomial number of elementary computational steps.

the graph of the traveling salesman problem

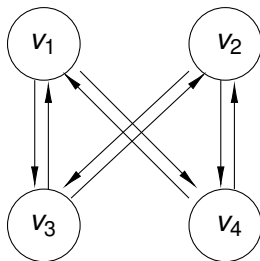
Given a graph $G = (V, E)$ for which we want to find a Hamiltonian cycle, construct a weighted directed graph G' as follows.

- If $(u, v) \in E$, then set $d(u, v) = 1$ in G' .
- If $(u, v) \notin E$, then set $d(u, v) = 2$ in G' .

The construction of G' is linear in the number of edges of G , or equivalently, is $O(n^2)$, for n vertices.

Hamiltonian cycle as a traveling salesman problem

Exercise 4: Encode



as a traveling salesman problem. Demonstrate that the solution of the traveling salesman problem solves the Hamiltonian cycle problem.

the proof follows from the lemma

Lemma (8)

*G has a Hamiltonian cycle if and only if
 G' has a tour of length at most n .*

Proof. If and only if means $\Rightarrow$ and $\Leftarrow$.

- $\Rightarrow$ If G has a Hamiltonian cycle,
then listing the visited vertices in the order of the cycle represents
a tour of length n , as every edge in G' has length 1.
- $\Leftarrow$ If G' has a tour of length at most n ,
then we show that the graph G has a cycle. Because the tour
visits all n vertices and the total distance is at most n , the sum of
all distances must be a sum of all ones, otherwise the tour could
not visit all n vertices.
So all visited edges on the tour in G' are edges in G .
The tour returns to the start vertex, so we have a cycle in G .

Q.E.D.

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

the Hamiltonian Path Problem

Definition (9)

Given a directed graph $G = (V, E)$, a path P in G is a *Hamiltonian path* if P contains each vertex exactly once.

In contrast with a Hamiltonian cycle, the path does not return to the start vertex.

The Hamiltonian Path Problem asks

Given a directed graph G , does G contain a Hamiltonian path?

Hamiltonian path is in $\mathcal{NP}$

Lemma (10)

The Hamiltonian path problem is in $\mathcal{NP}$.

Proof. A certificate for the Hamiltonian path problem is a path P .

A certifier algorithm verifies that

- P is indeed a path in the graph, that all vertices listed on P are connected by an edge, and
- that all vertices in the graph occur exactly once in P .

The certifier runs in time $O(nm)$,
for a graph with n vertices and m edges.

Thus the Hamiltonian path problem has an efficient certifier.

Q.E.D.

Hamiltonian path is NP-complete

Theorem (11)

The Hamiltonian Path Problem is NP-Complete.

Proof. By Lemma (10), Hamiltonian path problem is in $\mathcal{NP}$.

We will show that Hamiltonian Cycle $\leq_P$ Hamiltonian Path.

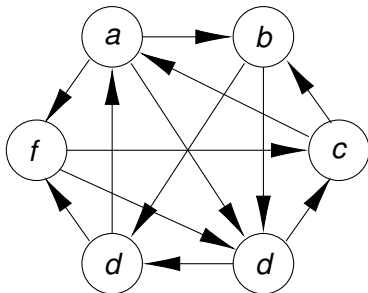
Consider an arbitrary instance of the Hamiltonian Cycle Problem given by the graph $G = (V, E)$.

Construct a graph $G' = (V', E')$ by selecting some $v \in V$ and replacing v by v' and v'' in V' .

- For every $(v, w) \in E$ by $(v', w) \in E'$.
- For every $(u, v) \in E$ by $(u, v'') \in E'$.

Hamiltonian Cycle as Hamiltonian Path problem

Encode



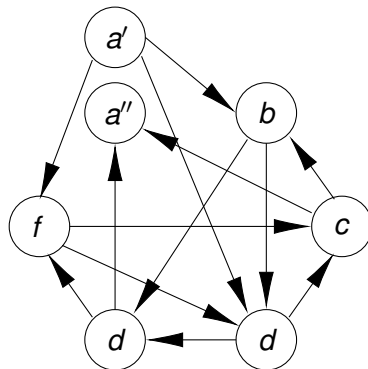
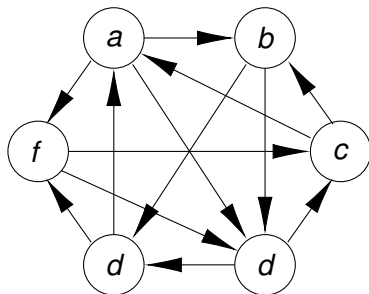
as a Hamiltonian path problem. Demonstrate that the solution of the Hamiltonian path problem solves the Hamiltonian cycle problem.

Why is this a polynomial-time reduction?

encoding Hamiltonian Cycle

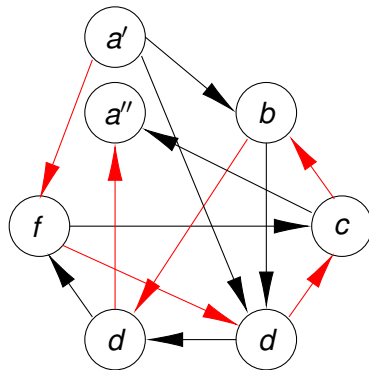
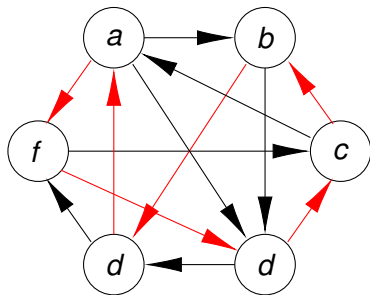
We have a cycle starting at a and ending at a .

Replacing a by a' and a'' in the graph at the left gives the graph at the right:



equivalent solutions

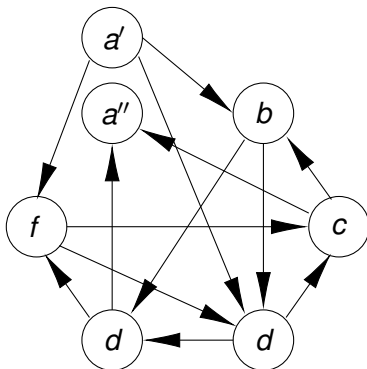
We have a cycle starting at a and ending at a , shown at the left:



and, at the right, we have a Hamiltonian path from a' to a'' .

a polynomial-time reduction

The graph G' derived from G , input to Hamiltonian cycle



has one extra vertex and the same number of edges as G .
 G' is constructed in $O(1)$ time, which is polynomial time.

the theorem follows from the lemma

Lemma (12)

G' has a Hamiltonian path if and only if G has a Hamiltonian cycle.

Proof. If and only if means $\Rightarrow$ and $\Leftarrow$.

- $\Rightarrow$ If G' has a Hamiltonian path P ,
then the end points of the path are v' and v'' .
If we replace v' and v'' with v ,
then we obtain a Hamiltonian cycle in G .
- $\Leftarrow$ If G has a Hamiltonian cycle C ,
consider starting and ending C with v .
Replacing v with v' and v'' ,
the same cycle C defines a Hamiltonian path P in G' .

Q.E.D.

two NP-complete sequencing problems

Is there a cycle visiting each vertex in a directed graph exactly once?

Theorem (5)

The Hamiltonian Cycle Problem is NP-complete.

An arbitrary instance of 3-SAT is a Hamiltonian cycle problem.

Given weighted, directed graph G , and a bound D on the distance traveled, is there a tour in G of length at most D ?

Theorem (7)

The traveling salesman problem is NP-complete.

An arbitrary instance of Hamiltonian cycle is traveling salesman.

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

CS 401/MCS 401 Lecture 20
Computer Algorithms I
Jan Verschelde, 3 August 2026

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

the subset sum problem

A decision version of the subset sum problem is

Given natural numbers $w_1, w_2, \dots, w_n$ and W ,
is there a subset $S \subseteq \{w_1, w_2, \dots, w_n\}$ so that $\sum_{w \in S} w = W$?

With dynamic programming we obtained an $O(nW)$ algorithm,
with a pseudo-polynomial running time.

But W is given as a bit sequence

$$c_k 2^k + \dots + c_2 2^2 + c_1 2 + c_0, \quad c_i \in \{0, 1\}.$$

The size of W grows exponentially with k , the size of the input.

solve the subset sum problem

The subset sum problem is important.

Exercise 5: Solve the subset sum problem by giving an algorithm to find a subset whose weights sums up to the given W .

What is the worst case running time of your algorithm?

subset sum is in $\mathcal{NP}$

Lemma (6)

The subset sum problem is in $\mathcal{NP}$.

Proof. The certificate consists of the subset $S \subseteq \{w_1, w_2, \dots, w_n\}$.

The certifier computes $\sum_{w \in S} w$ and checks whether this sum equals W .

The computation of the sum and the check require $O(n)$ operations.

So we have an efficient certifier and subset sum is in $\mathcal{NP}$.

Q.E.D.

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- **reducing 3-dimensional matching to subset sum**
- a partial taxonomy of hard problems

3-dimensional matching is NP-complete

Given are the following:

- Three sets X , Y , and Z :
 - 1 each set has the same size $n = \#X = \#Y = \#Z$, and
 - 2 the sets are disjoint: $X \cap Y = \emptyset$, $X \cap Z = \emptyset$, $Y \cap Z = \emptyset$.
- A set T of ordered triples $T \subseteq X \times Y \times Z$.

The 3-dimensional matching problem asks

Is there a set of n triples in T so each element of $X \cup Y \cup Z$ is contained exactly once in one of these triples?

Such a set of triples is *a perfect three-dimensional matching*.

Example: $X = \{\text{instructors}\}$, $Y = \{\text{courses}\}$, $Z = \{\text{times}\}$.

reducing 3-dimensional matching to subset sum

To prove that subset sum is NP-complete, we will show that

$$\text{3-Dimensional Matching} \leq_P \text{Subset Sum}.$$

Any instance of the 3-dimensional matching problem is given by

- 3 sets X , Y , and Z , all of size n , and
- a set of m triples $T \subseteq X \times Y \times Z$.

A set can be represented as a bit vector

- bit i is 1 if the i -th element belongs to the set,
- bit i is 0 otherwise.

Any bit vector can be seen to hold the coefficients of a number.

A triple $t = (x_i, y_j, z_k)$ is represented by a bit vector of size $3n$:

- 1 for x_i , we have a 1 at position i ,
- 2 for y_j , we have a 1 at position $n + j$,
- 3 for z_k , we have a 1 at position $2n + k$.

an example

triple t_i			x_1	x_2	x_3	x_4	y_1	y_2	y_3	y_4	z_1	z_2	z_3	z_4	w_i	
x_1	y_2	z_3	1	0	0	0	0	1	0	0	0	0	0	1	0	100,001,000,010
x_2	y_4	z_2	0	1	0	0	0	0	0	0	1	0	1	0	0	10,000,010,100
x_1	y_1	z_1	1	0	0	0	1	0	0	0	1	0	0	0	0	100,010,001,000
x_2	y_2	z_4	0	1	0	0	0	1	0	0	0	0	0	0	1	10,001,000,001
x_4	y_3	z_4	0	0	0	1	0	0	1	0	0	0	0	0	1	100,100,001
x_3	y_1	z_2	0	0	1	0	1	0	0	0	0	1	0	0	0	1,010,000,100
x_3	y_1	z_3	0	0	1	0	1	0	0	0	0	0	1	0	0	1,010,000,010
x_3	y_1	z_1	0	0	1	0	1	0	0	0	1	0	0	0	0	1,010,001,000
x_4	y_4	z_4	0	0	0	1	0	0	0	1	0	0	0	0	1	100,010,001
															111,111,111,111	

Observe:

We have a 3-dimensional matching if and only if the sum of some subset is $W = 111,111,111,111$.

constructing the weights

The m triples in T define bit vectors of size $3n$.

We defined $3nm$ bits, a polynomial number in the dimensions n and m of the 3-dimensional matching problem.

For base B , the triple $t = (x_i, y_j, z_k)$, defines the number

$$w_t = B^{i-1} + B^{n+j-1} + B^{2n+k-1}.$$

The adding of numbers corresponds to taking unions of triples, *provided there is no carry over when adding numbers.*

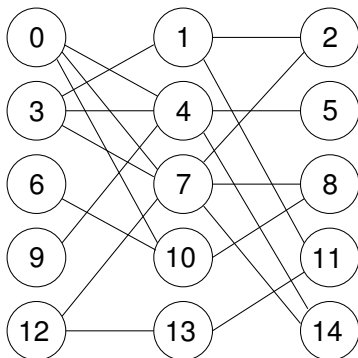
We prevent carry over, if the base is large enough. Take $B = m + 1$.

Define W as a number in base $m + 1$ with $3n$ digits, all equal to one:

$$W = \sum_{i=0}^{3n} (m+1)^i.$$

encode 3-dimensional matching into subset sum

Exercise 6: Consider the 3-dimensional matching problem, given by $X = \{ 0, 3, 6, 9, 12 \}$, $Y = \{ 1, 4, 7, 10, 13 \}$, $Z = \{ 2, 5, 8, 11, 14 \}$.



- 1 Construct the weights for the subset sum problem.
- 2 Demonstrate that the solutions to the two problems are equivalent.

3-dimensional matching is equivalent to subset sum

Lemma (7)

The set T of triples contains a perfect 3-dimensional matching if and only if there is a subset of the weights that adds up to W .

Proof. If and only if means $\Rightarrow$ and $\Leftarrow$.

- $\Rightarrow$ If we have a perfect 3-dimensional matching, then in the sum of the weights, there is a single one at each position in the $3n$ -digit number, so the result equals W .
- $\Leftarrow$ If we have a subset of weights $w_{t_1}, w_{t_2}, \dots, w_{t_k}$ that adds up to W , then since each w_{t_ℓ} has three ones in its representation and there are no carry overs, so $k = n$.

For each of the $3n$ positions, exactly one of the w_{t_ℓ} has a one in that position. Therefore, the $t_1, t_2, \dots, t_k$ define a perfect matching.

Q.E.D.

NP-completeness

Theorem (8)

The subset sum problem is NP-complete.

Proof. By Lemma (6), the 3-dimensional matching problem is in $\mathcal{NP}$.

An arbitrary instance of the 3-dimensional matching problem can be reduced in polynomial time to a subset sum problem.

By Lemma (7), the reformulated 3-dimensional matching problem can be solved by the subset sum problem.

Q.E.D.

Scheduling with Release Times and Deadlines

We want to schedule n jobs on a single machine.

The input for each job i , $i = 1, 2, \dots, n$, has three numbers:

- 1 r_i is the release time, when it is available for processing,
- 2 t_i is the processing duration, and
- 3 d_i is the deadline by which it must be completed.

Can we schedule all jobs so that each completes by its deadline?

The subset sum problem can be reduced in polynomial time to scheduling with release times and deadlines.

Theorem (9)

Scheduling with release times and deadlines is NP-complete.

Sequencing & Numerical Problems

1 The Hamiltonian Cycle Problem

- computing Hamiltonian cycles in a graph
- representing assignments with cycles
- Hamiltonian cycle is NP-complete

2 The Traveling Salesman Problem

- a decision problem
- traveling salesman is NP-complete
- the Hamiltonian Path Problem

3 Numerical Problems

- the subset sum problem
- reducing 3-dimensional matching to subset sum
- a partial taxonomy of hard problems

a partial taxonomy of hard problems

Circuit satisfiability was the first NP-complete problem.

Then we proved that 3-SAT is NP-complete.

① packing and covering problems

Independent Set, Set Packing, Vertex Cover, Set Cover

② sequencing problems

Hamiltonian Cycle, Traveling Salesman

③ partitioning problems

3-Dimensional Matching, 3-Coloring

④ numerical problems

Subset Sum, Scheduling with Release Times and Deadlines

a very short summary of the first four chapters

- 1 An algorithm is an ordered set of unambiguous, executable steps that define a terminating process.
The Gale-Shapley algorithm solves the stable matching problem.
- 2 An algorithm is efficient if its worst case asymptotic cost is bounded by a polynomial in its input size. Implementing an algorithm determines running times of operating data structures.
- 3 Graph traversals have a running time of $O(n + m)$ for n vertices and m edges, which is also the cost of topological ordering of a directed acyclic graph.
- 4 Greedy algorithms solve optimization problems, such as interval scheduling, caching, shortest paths, spanning trees, via locally optimal rules.

a very short summary of the next four chapters

- 5 Divide and conquer methods applied to counting inversions, closest pairs of points, integer multiplication, convolutions, and matrix multiplications give subquadratic and subcubic times.
- 6 Dynamic programming derives a polynomial number of subproblems which joint solutions solve the original problem with a recurrence, applied to weighted interval scheduling, segmented least squares, sequence alignment, subset sum, knapsack, shortest paths in directed graphs with negative weights.
- 7 The Ford-Fulkerson algorithm computes maximum flow in pseudo-polynomial time and solves bipartite matching, circulation with demands, edge-disjoint paths, survey design, and airline scheduling.
- 8 If a polynomial time algorithm existed for an NP-complete problem, then all problems for which we have an efficient certifier could be solved in polynomial time.